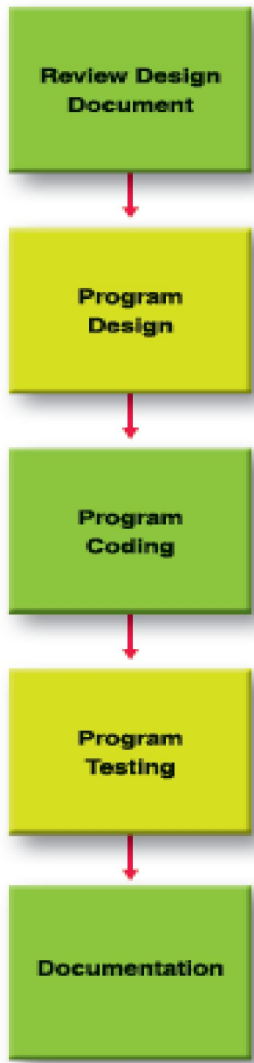


Chapter 9

System Development 1 – Programming and Tools

Program Development Stages



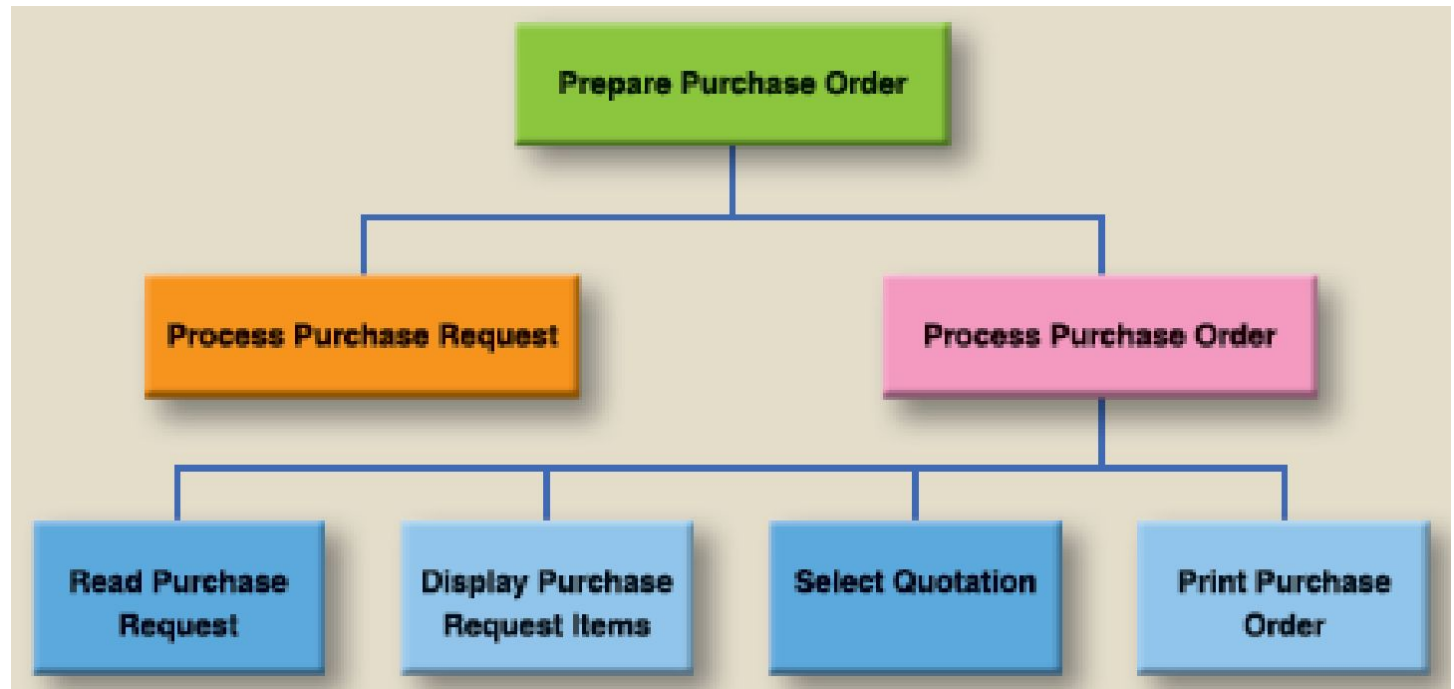
- a. Documentation - DFD, process description, screen layouts, report layout, source documents, data dictionary.
 - b. Discuss any changes with S.A. (who review with end-users).
-
- a. Use techniques (decision tables, program flowchart, Structured English, structure charts).
 - b. Use tools (eg. CASE tools).
-
- a. Writing program instructions.
 - b. Use of tools, CASE, 4GL, report writers, screen generators, program generators.
-
- a. Various software testings are carried out eg. unit, integration, system, user acceptance.
-
- b. 4 documentation : end-user, operation, program and system.

Program Design

Program Design

- **Reviewing.** The Program Specification is reviewed to see whether any changes needed.
- **Design Tools.** The programmer typically uses one of several program design tools to aid in logic development eg. structure charts, flowcharts, and pseudocode

Structure Chart



Structured Programming

- Definition

- Translating the system design into **well-structured** program modules, making them more readable, understandable, maintainable.

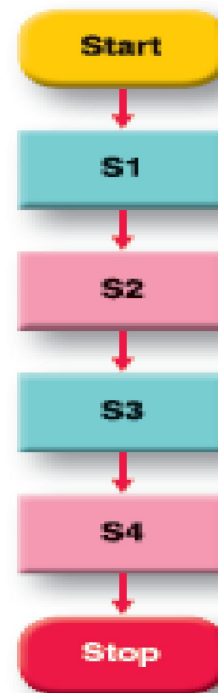
- Historical Development

- **1960s** – to make the system more *readable and understandable* codes.
- **Early 1970s** – used *top-down* partitioning to make the system more *modular* etc to design flexible and maintainable systems
- **Mid 1970s** – Good programs to wrong problems (ie. not *meeting requirements*). Little input from users.

3 Basic Control Structures

1. Sequence Structure

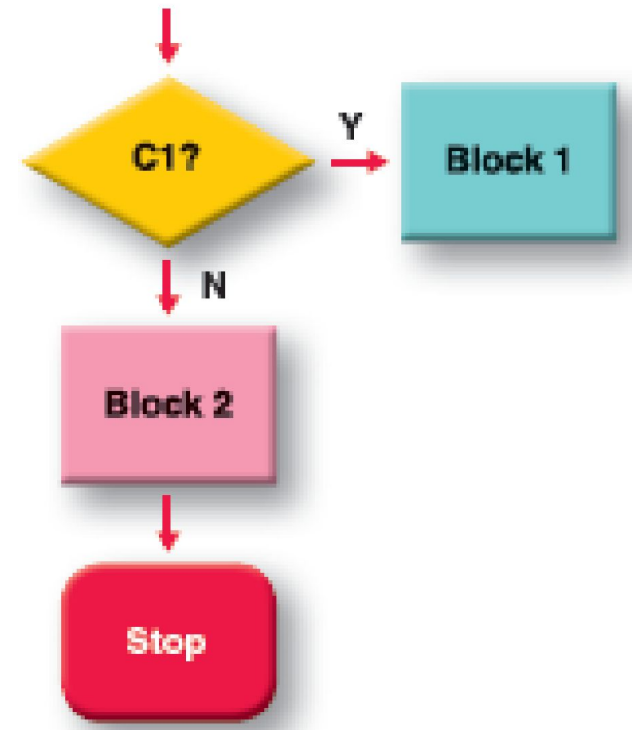
- Sequence logic executes the program statements in the order in which they appear.
 - Do action 1 (S1)
 - Do action 2 (S2)
 - Do action 3 (S3)
 - Do action 4 (S4)



2. Selection / Decision Structure

• Example

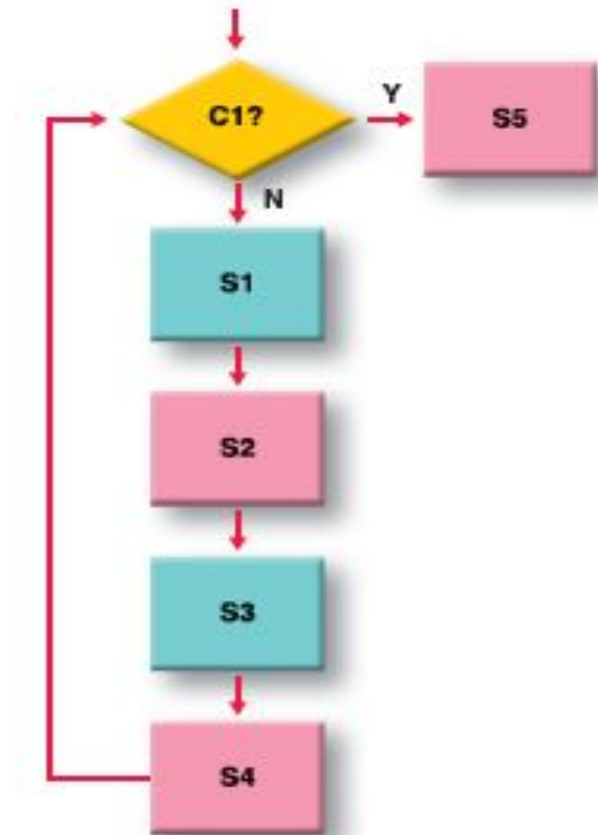
- **IF** condition 1 (C1) is true
- **DO** action(s) in block 1
- **ELSE** (if condition 1 is untrue)
- **DO** action(s) in block 2
- **ENDIF**



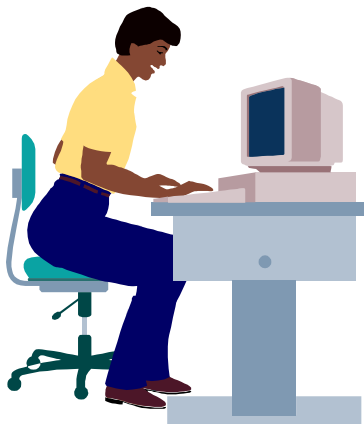
3. Repetition / Looping Structure

- Example

- DO WHILE condition 1 (C1) is true
- action(s)
- ENDDO



Code the Program



Coding Program

- **Purpose** - Process of writing the program instructions that implement the program specifications, using a suitable language (such as C++).
- **Program design** - A good program design, the coding process is greatly simplified.
- **Large system** - The various modules might be subdivided into individual **sub-modules** and each assigned to a **number** of programmers.

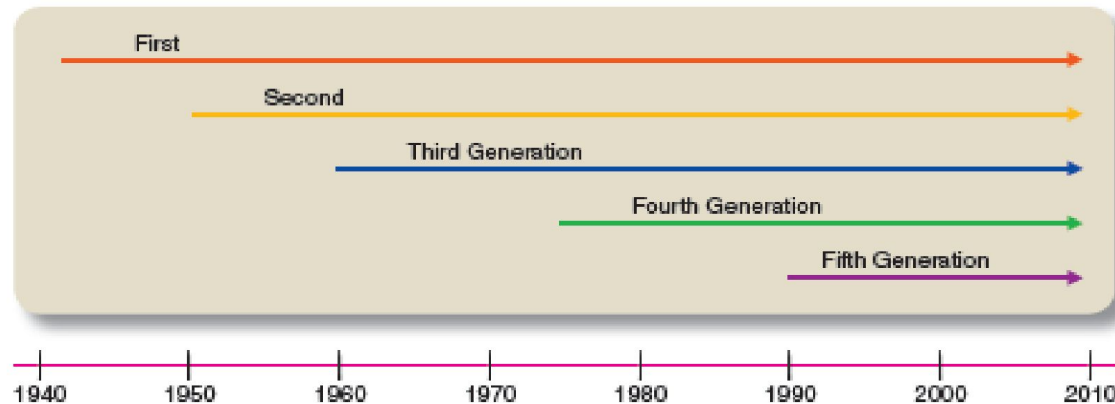
Criteria for Choosing a Programming Language

- **Suitability of Language.** Different languages are suitable for different applications and business environments eg. C and C++ for system programming; Jscript, C# and HTML for embedded web-based applications.
- **Familiarity with Language.** Programmers' familiarity with a particular programming language.
- **Features of Language.** Supports a top down design or modular programming approaches and structured programming concepts.

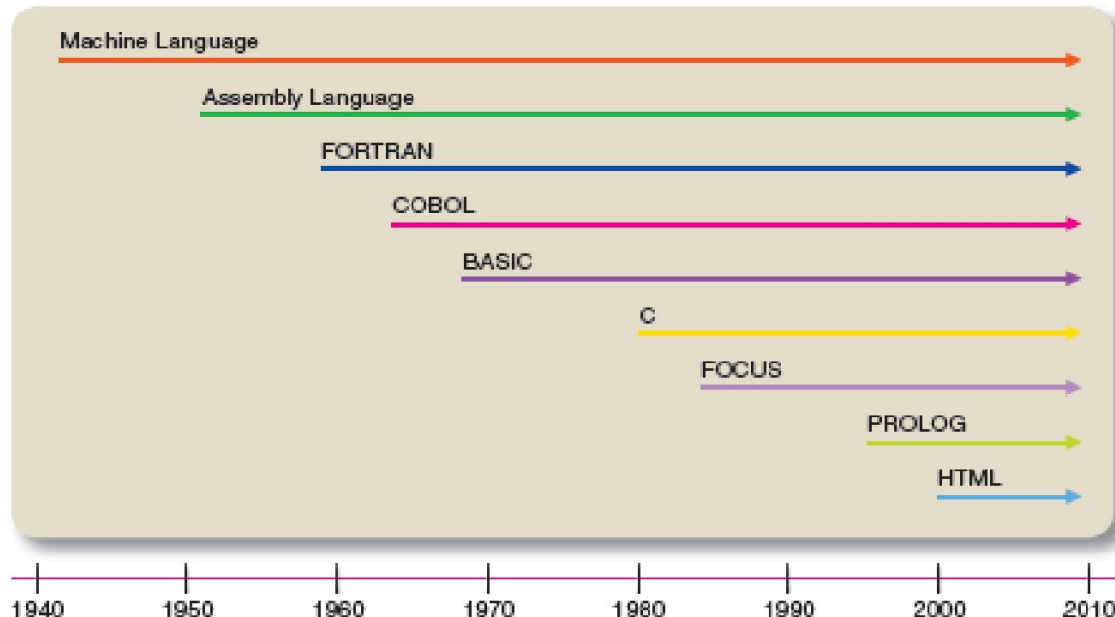
Criteria for Choosing a Programming Language ... (cont)

- **Cost.** Cost elements may include the purchase costs, license fees which depends on the number of users, and any hardware that needs to be purchased.
- **Support Tool.** How easy a programming language can be used or learnt can affect the productivity of a programmer.
- **Performance.** Refers to efficiency of programs during execution or whether they would be resource hungry and take up large memory space.

Programming Languages



Generation of Languages



Specific Languages

Generation or Levels of Languages

No	Language	Generation	Abbreviation
1	Machine Language	First Generation	1GL
2	Assembly Language	Second Generation	2GL
3	High-level language	Third Generation	3GL
4	Very high-level language	Fourth Generation	4GL
5		Fifth Generation	5GL

1GL

- A machine language in 0s and 1s
- Example : 00100001

0000DE	5A50	35AA	
0000E2	47F0	2100	00102
000104	5870	304E	01050
000108	1C47		0010D8

2GL

- An assembly language in mnemonic codes
- Example : **ADD AX,8**

*	THIS MODULE CALCULATES THE TIME PAY	
CALCSTPY	EQU	*
	ST	14, SAVERTPY
	SR	4, 4
	SR	7, 7
	SR	5, 5

3GL

- A 'high-level' programming language.
- Example : C++, Java

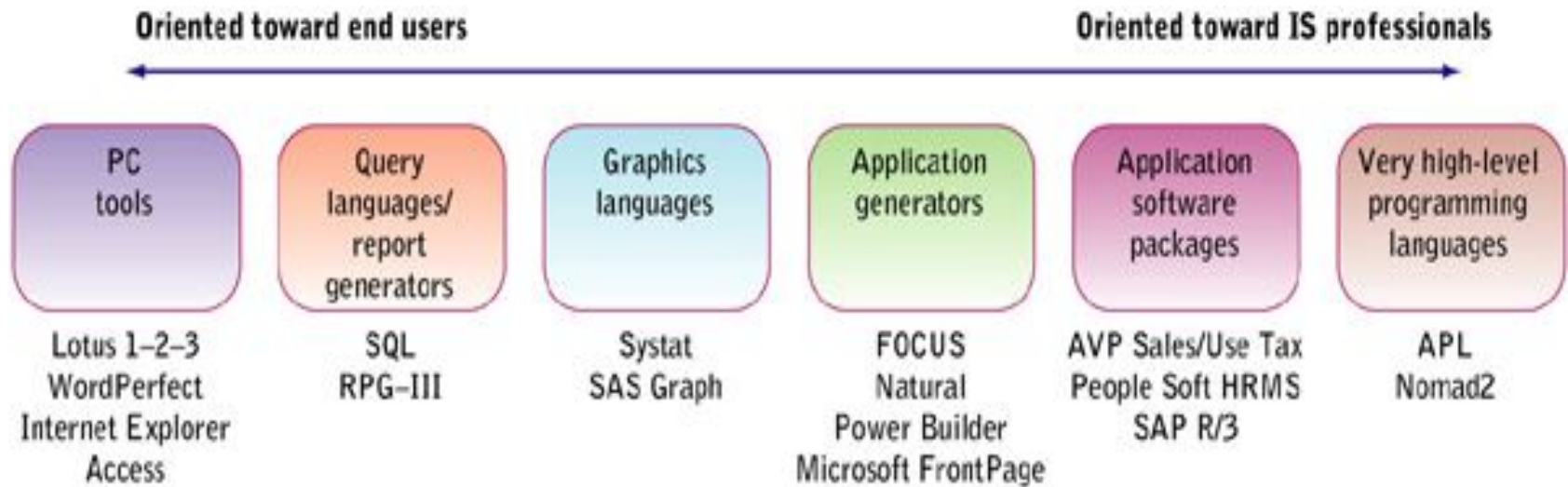
* COMPUTE REGULAR TIME PAY
MULYIPLY REGULAR-TIME-HOURS BY HOURLY-PAY-RATE
GIVING REGULAR-TIME-PAY
* COMPUTE OVERTIME PAY
IF OVERTIME-HOURS > 0
COMPUTE OVERTIME-PAY = OVERTIME-HOURS * 1.5 * HOURLY-PAY-RATE
ELSE
MOVE 0 TO OVERTIME-PAY

Fourth Generation Languages (4GLs)

Main Features of 4GL

- **User-friendly.** More English-like. Enable end-users to develop their own software applications.
- **Programming tools.** Variety of software tools such as application and report generators. Increase productivity of programmers.
- **Non-procedural.** Most 4GLs are nonprocedural languages that encourage users and programmers to use.

Examples of 4GLs



Examples of 4GLs

- **Query Language.** Used to request the system to display reports (ad hoc) to a specific *query* fast.
- **Report Generator.** User can design a well-formatted *report* layout (routine).
- **Application Generator.** Allows you to specify the operations and the screen layout. It automatically generates the *source codes*.

Advantages of 4GLs

- **End-users involvement.** Brings commitment.
- **Easy maintenance.** Maintenance is simpler because application development time is rapid.
- **Cost-effective.** Use less time. However, 4GLs are slower than conventional programming languages and require much more memory eg. C++
- **Generate documentation.** Documentation is produced at the same time as the prototype system. System should be easier to maintain.

Exercise for Students

- **ABC Ltd is a multinational organisation which has its headquarters in US and overseas operations in many parts of the world.**
- **XYZ Ltd is one of the many operations in Malaysia. Explain 3 factors to consider when selecting a programming language.**

Suggested Answers (brief)

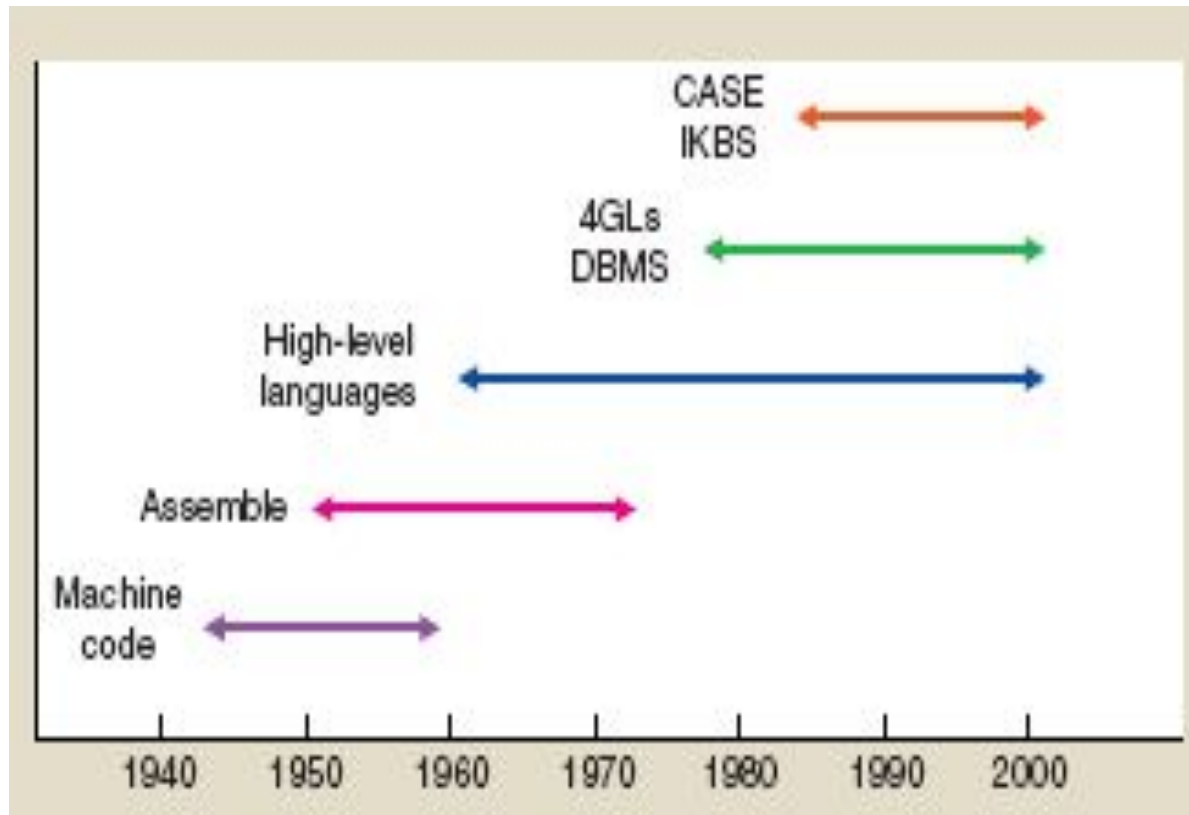
- The programming language used for existing applications – for **compatibility**.
- The **standards** imposed by the headquarters in US.
- The **nature** of the application to be developed.
- The **familiarity** of a programming language by the programming team.

CASE Tools

Definition of CASE Tools

- **Acronym.** Computer aided software engineering.
- **Software.** A piece of software that allows the analyst to develop and maintain the system more efficiently.
- **Purpose.** Automating development tasks to help in improving analyst *productivity* and systems *quality*.
- **Examples.** Draw diagrams like DFD, automatic file and program development facilities.

Historical Development



CASE -
Compu^{re}-Aided
Software Engineering

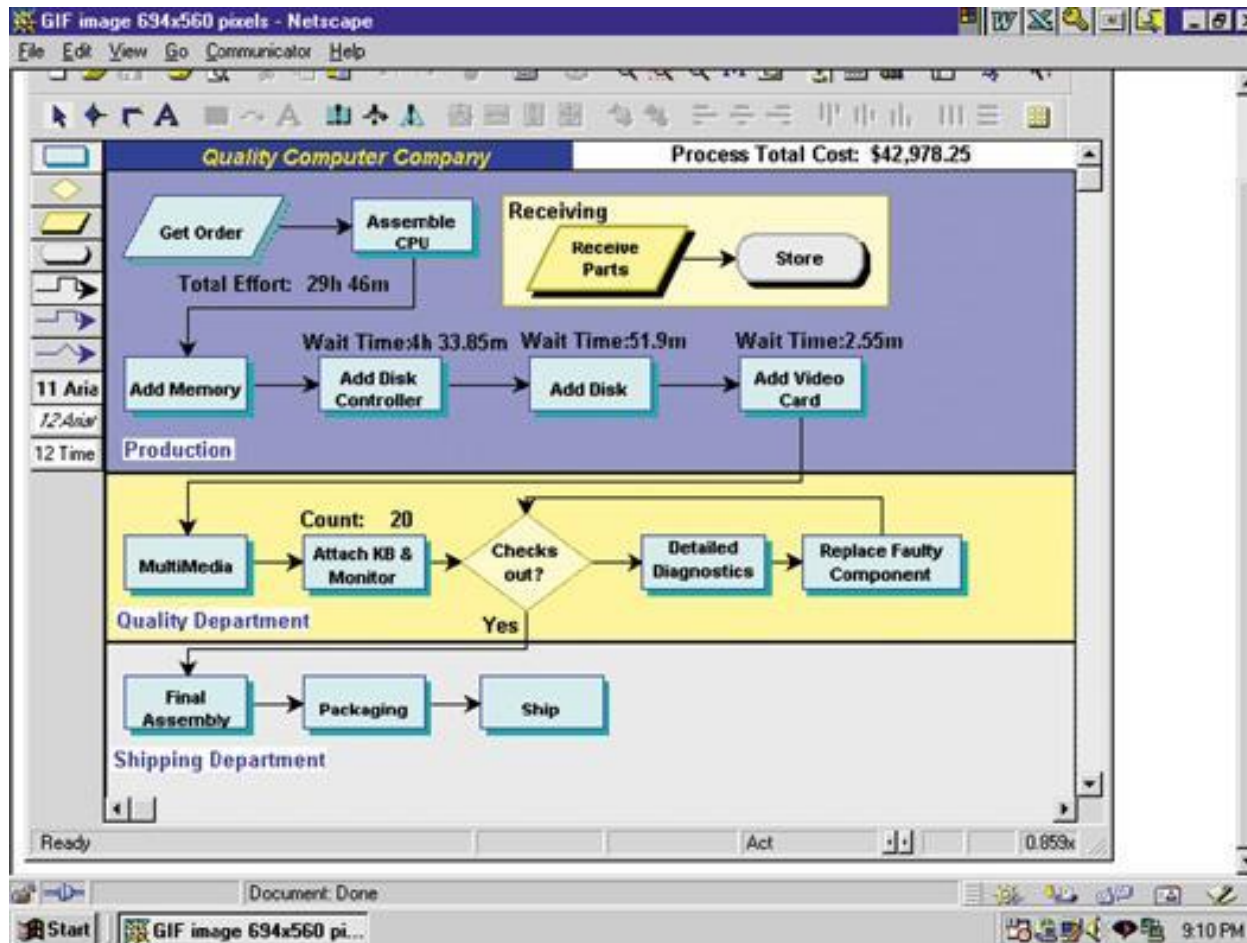
IKBS -
Intelligent Knowledge
Based System

Examples of CASE Tool

- Powersoft's S-Designer
- Sybase's Power Designer/Builder
- Oracle System's CASE Designer 2000
- Logic Works' Erwin (data modeling, client/server applications)
- IBM's Rational Software Architect
- KnowledgeWare's Application Development Workbench (ADW) and IEW

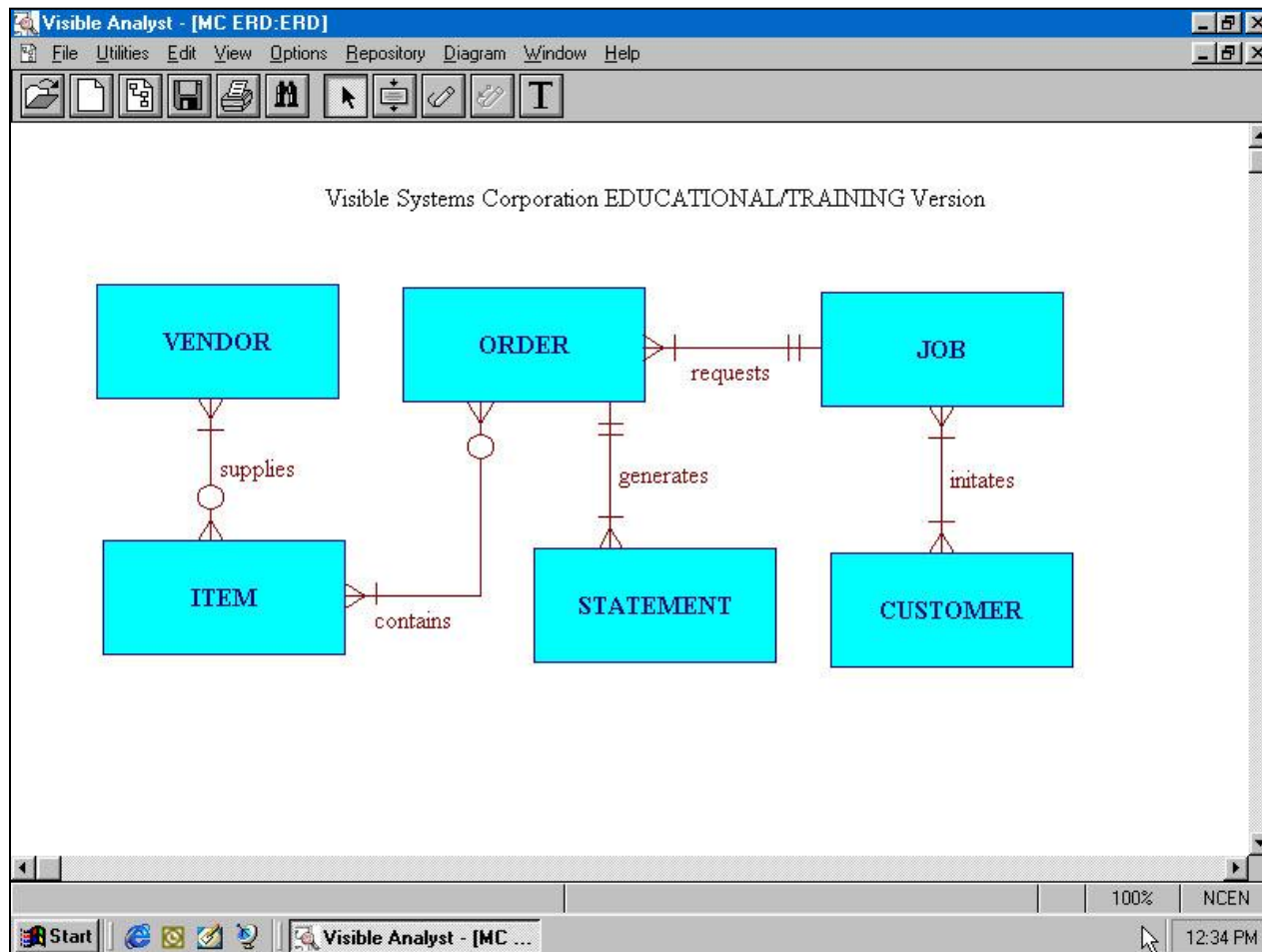
CASE Tools – Example 1

Scitor's Process 98



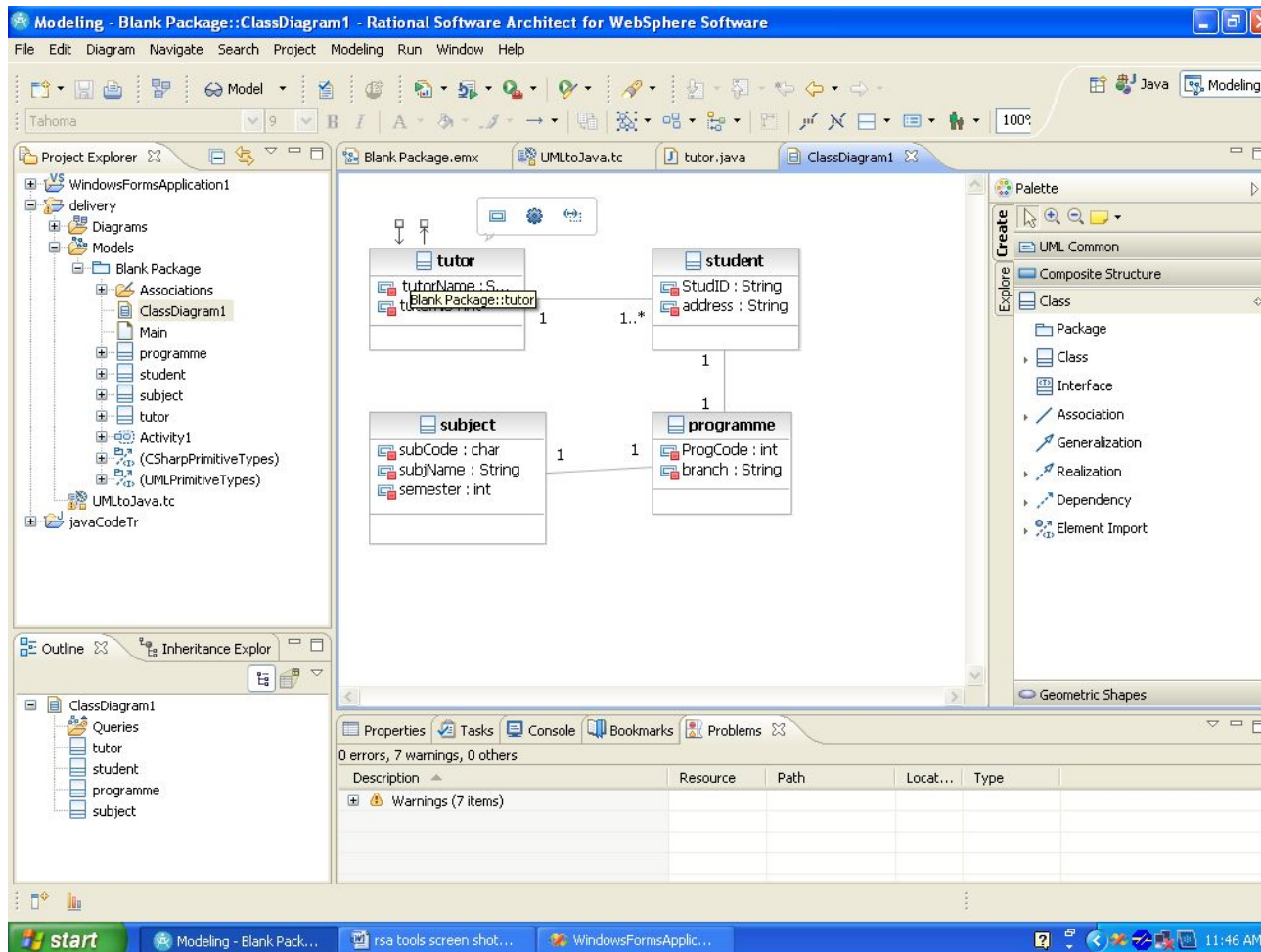
Case Tools – Example 2

- Visible Systems Corporation's **Visible Analyst** Workbench



CASE Tools – Example 3

- IBM's Rational Software Architect



CASE

Terms and Concepts

CASE Terms and Concepts

- **Modeling Tools**
 - Used to model system graphically
 - Examples : DFDs, UML diagrams, structure charts etc
- **Documentation Tools**
 - The **repository** automatically identifies the new entries and adds them to the database.
 - CASE tools **check** automatically for inconsistent or incomplete information in forms, reports, and diagrams.
- **Engineering Tools**
 - **Forward engineering** means translating business processes and functions into applications by editing objects and code directly, or by modifying graphical representations such as DFDs and UML diagrams.
 - **Reverse engineering** allows you to examine an existing application and break it down into a series of diagrams, structure charts, and, in some cases, source code.

Development Tools

- **Screen generator**

- An interactive tool that helps you design a custom interface, create screen forms, and handle data entry format and procedures.
- Control how the screen will display captions, data fields, data, and other visual attributes.

- **Report generator**

- **Designs** formatted reports rapidly
- Can input **sample field values** to create a mock-up report for users to review and approve (as in prototyping).
- **Creates** a report definition and program code that actually produces the report.
- **Modifies** a report easily at any stage of the design process.

- **Application generator**

- Develops computer **programs** rapidly by translating a logical model directly into code.
- Uses information stored in the data repository to generate applications in 4GLs such as Java, C++, and Visual Basic.

Roles / Benefits of Using CASE Tools in Development and Maintenance of Models

1. Drawing Models

1. **Easy to draw models.** Standard shapes can be selected provide professional looking diagrams.
2. **Easy to store models.** Allow subsequent retrieval and amendment.
3. **Easy to update models.** Models are easy to be edited, updated and re-printed.

2. **Enforcing standards.** Ensuring that models follow defined standards and conventions eg. certain relationship not allowed.

Roles / Benefits of Using CASE Tools in Development and Maintenance of Models

3. **Automatic cross-referencing** between different models (eg. ERD and ELH) to promote a consistent view of the system under development.
4. **A central Data Dictionary.** Hold the **data elements** within each entity (eg. format, value range, default values etc.) in *one place* and make them easy to maintain and update.
5. **Automatic File / Program framework.** Automatically produce physical file and data structures from the entities defined in the entity-relationship model.

Exercise for Students

- **Mr. Leong, a software development consultant, proposed the use of CASE tools in drawing and amending diagrams eg. DFD, ERD, UML diagrams.**
- **Use 3 reasons to justify his proposal.**

Advantages of Using CASE tool in System Development

- 1. Draw descriptive diagrams.** Diagrams (DFD, ERD) can be drawn easily with standard shapes available.
- 2. Producing and maintaining diagrams.** Graphical editing facilities. Easy changes to models can be made.
- 3. Adhering to development standards.** Force IT personnel to adhere to standards eg. a data store cannot be directly linked to an external entity in DFD.

Advantages of Using CASE tool in System Development (cont...)

4. Maintaining a data dictionary

- Stores information about the constituent parts of systems specification. Eg. entries for dataflows, data stores, entities, processes, entities.**
- Supports the consistency and cross-referencing checks eg. the data stores of the DFD to the entities of ERD.**

5. Prototyping. Develop screens, reports and other layouts. Contents can be displayed and screens can be linked.

6. Generating Computer Codes. Converts systems specifications into executable source code and entities into physical files and databases.

Toolsets To Support Software Development

- **Integrated Development Environments (IDE)**

- **Definition**

- A software that provides comprehensive facilities or tools to programmers to assist them in software development eg. CASE tools and tools for construction of GUI, class hierarchy diagram.

- **Advantages**

- Maximise programmer productivity by providing tightly-knit components with similar user interfaces. A programmer has much less mode switching to do.
 - The many tools makes it easier to design, develop and maintain systems.

- **Disadvantage**

- IDE is a complicated piece of software, the high productivity only occurs after a lengthy learning process.

- **Examples of IDE**

- **Microsoft Visual Studio**

- **IBM's Rational Software Architect**